



Concurrency Control (2) - timestamp

There are many techniques to ensure concurrency control, we already discussed lock-based protocol

- idea is to achieve conflict serializable schedule without using lock.

✓ Timestamp-Based Protocols

Timestamp of transaction

- Each transaction T_i is issued a timestamp $TS(T_i)$ when it enters the system.
 - Each transaction has a *unique* timestamp
 - Newer transactions have timestamps strictly greater than earlier ones
 - Timestamp could be based on a logical counter
 - Real time may not be unique
 - Can use (wall-clock time, logical counter) to ensure
- Timestamp-based protocols manage concurrent execution such that **time-stamp order = serializability order**
- Several alternative protocols based on timestamps

- each transaction you have unique timestamp of each transaction (like the process number)
- Newer transactions have timestamps greater than early one
- the timestamp number for each transaction generated by DBMS,
- when you transport to other system, you may have trouble because the precision of the clock is not the same.

Timestamp of data item



Timestamp-Ordering Protocol

The **timestamp ordering (TSO)** protocol

- Maintains for each data Q two timestamp values:
 - **W-timestamp**(Q) is the largest time-stamp of any transaction that executed **write**(Q) successfully.
 - **R-timestamp**(Q) is the largest time-stamp of any transaction that executed **read**(Q) successfully.
- Imposes rules on read and write operations to ensure that
 - Any conflicting operations are executed in timestamp order
 - Out of order operations cause transaction rollback

- each data item Q
 - imagine Q as a row → when you read or write Q , the system maintain W-timestamp, and R-timestamp
 - largest timestamp of any transaction that executed write(Q) successfully

Timestamp based technique

- For each transaction T_i , a timestamp $TS(T_i)$ is assigned to the transaction when it is started
- if T_j is younger than T_i , then $TS(T_i) < TS(T_j)$
- For each data item Q , $WTS(Q)$ which is the timestamp of the youngest (highest timestamp) transaction that has written Q successfully
 - the last one who wrote the value Q
- For each data item Q , there is a read time stamp $RTS(Q)$ which is the timestamp of the youngest transaction that read Q
 - the last one who read Q successfully, maybe not a last one who read

with this info, we can have algorithm to ensure conflict serializable schedule without using locking

Algorithm or protocol is call →

Timestamp-based Protocol

prevent incorrect result by rolling back

- there a set of rule and if your activity against the rule, you get rolled back (the correct sequence is checked)

Read



Timestamp-Based Protocols (Cont.)

- Suppose a transaction T_i issues a **read**(Q)
 1. If $TS(T_i) < W\text{-timestamp}(Q)$, then T_i needs to read a value of Q that was already overwritten.
 - Hence, the **read** operation is rejected, and T_i is rolled back.
 2. If $TS(T_i) \geq W\text{-timestamp}(Q)$, then the **read** operation is executed, and R-timestamp(Q) is set to $\max(R\text{-timestamp}(Q), TS(T_i))$.

1. If $TS(T_i) < W\text{-timestamp}(Q)$, then T_i needs to read a value of Q that was already overwritten.

- the value on that row was written by somebody younger than you (you can't read, overwritten already)
 - your transaction get rolled back

What if you want to write to row Q, but after the system check → the write timestamp of that row is higher than your transaction timestamp

- it's mean someone younger has already read the value → we are too slow, that somebody else suppose to read the value that we are about to write.
2. If $TS(T_i) \geq W\text{-timestamp}(Q)$, then the read operation is executed, and R-timestamp(Q) is set to " $\max(R\text{-timestamp}(Q), TS(T_i))$ "
 - the one who wrote value Q is older than us → we can read
 - or if equal → our transaction wrote the value → we can read
 - if we the youngest whoever read the value → r timestamp is us.

Write

- Suppose that transaction T_i issues **write**(Q).
 1. If $TS(T_i) < R\text{-timestamp}(Q)$, then the value of Q that T_i is producing was needed previously, and the system assumed that that value would never be produced.
 - Hence, the **write** operation is rejected, and T_i is rolled back.
 2. If $TS(T_i) < W\text{-timestamp}(Q)$, then T_i is attempting to write an obsolete value of Q.
 - Hence, this **write** operation is rejected, and T_i is rolled back.
 3. Otherwise, the **write** operation is executed, and $W\text{-timestamp}(Q)$ is set to $TS(T_i)$.

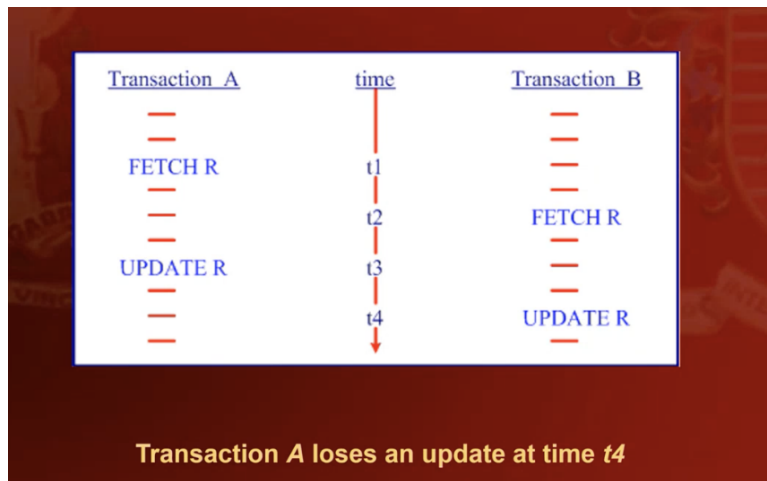
1. somebody else younger has read the value Q → the write operation is rejected, T_i is rolled back
2. You want to write, but somebody else younger has written the value → T_i is intent to write the obsolete value of Q → write operation is rejected, and T_i is rolled back
3. otherwise, write successfully → W-timestamp is us

if we apply read and write like above, it guarantee we obtain conflict serializable schedule

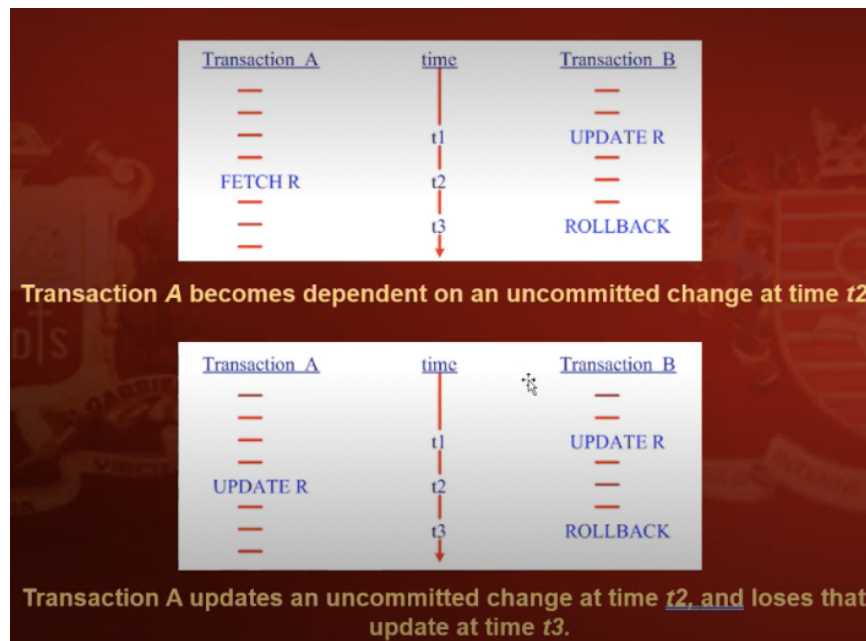
- this protocol has lot of rolled back → therefore not very popular, but lot of application use it, like the app where deadlock is has to be prevented (deadlock prevention)

Some Example

1. The lost update problem → what happen if we apply timestamp protocol to this schedule



- Assume that A is older than B (A come first)
 - timestamp of A is less than B
 - at time t_3 is the problem, A want to write R but R-timestamp of Q is greater than transaction A
 - TA is rolled back
 - TA want to write, but it was already read by someone younger
 - write rule 1
2. the uncommitted dependency rollback



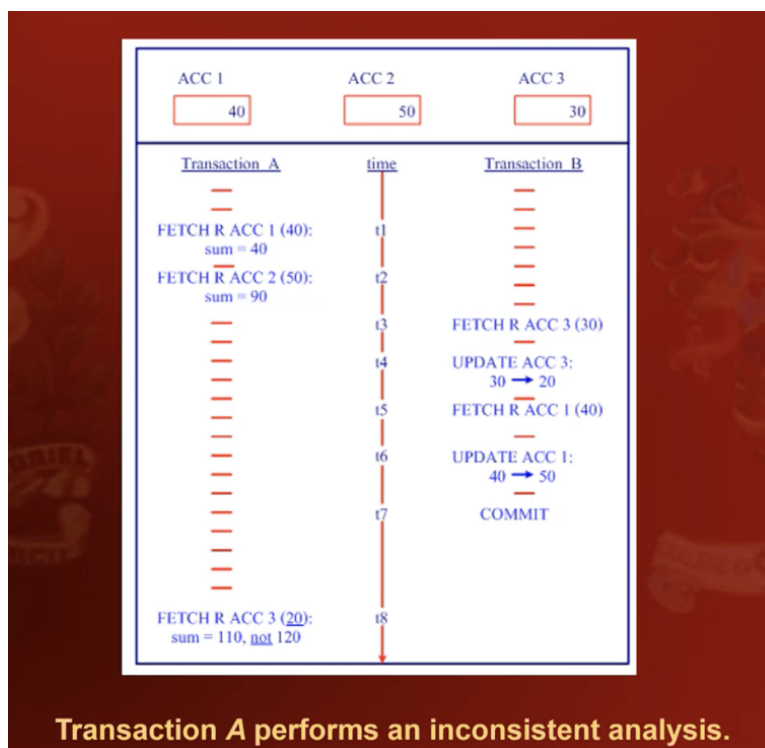
- the problem is it doesn't check whether any transaction has committed or not.

- like it roll back later
- the book suggested → commit bit
 - like if commit bit is on → allow to read
- in practice, commit bit is not practical → uncommit bit

idea

- what about only when we update → we issued an x lock, and unlock x lock at sync point, will it work
 - in this case, when you try to read, the value is not yet commit, wait until the transaction who wrote, commit and released the lock.
- if we introduced x lock to each update, will it lead to possible deadlock and why?
 - won't lead to deadlock because older transaction can't wait (timestamp protocol never allow older transaction to wait)

3. The inconsistent analysis problem



- at time t8, A will be rolled back
- because the write timestamp of acc3 is now b(younger than A)

- A can't read acc3, because it was written by someone younger
 - read number 1
4. phantom phenomena
- let said transaction B insert account 4, and A try to read account 4
 - it will get roll back → written by somebody else younger

What if we want correct result, no rolled back of transaction A in 3?

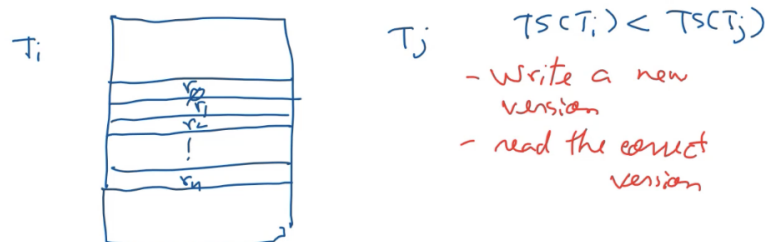
- The idea is to use older version, what if we have older version of account3, instead of read 20, read 30 → this is called multiversion scheme.
 - How many version can you afford to keep? a few
 - what if you don't have enough
 - read from log
 - Oracle, postgres used this
-

The timestamp ordering protocol may lead to too many rollbacks.

- To avoid many rollbacks, the multi-version scheme can be used.

This main memory → suppose r is a row

- when T_i want to read R, it get rolled back
- we can write a new r1, keep separate r1 row
- the idea is multiversion scheme
 - write a new version
 - read the correct version



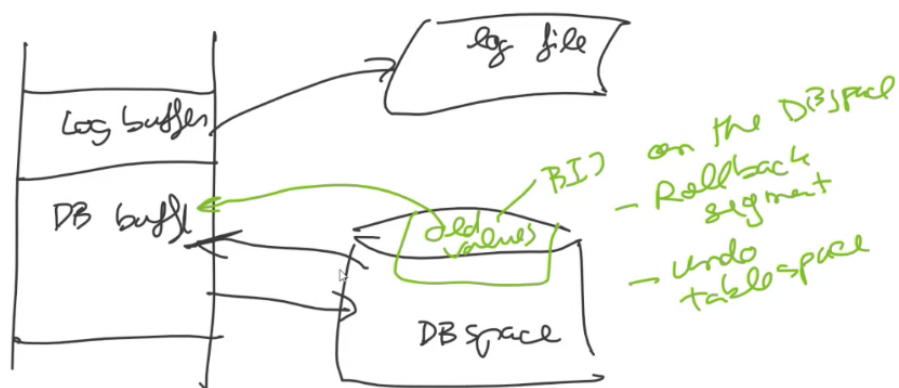
- r_0 is the youngest version which is written by a youngest transaction which is older than T_i

How many version should we kept (assume we have enough memory)

- if we know T_i will read T_0 , $r-1$, $r-2$ can go

What if T_i wants to read r but r_0 is not in the DB buffer?

- we read from log file? but actually we write from log buffer to log file, we don't read it other way around, unlike DBspace.
- this is why some implement BIJ on DBspace, not a separate log (older version of oracle called → rollback segment, undo tablespace)
 - older version can be obtain from here



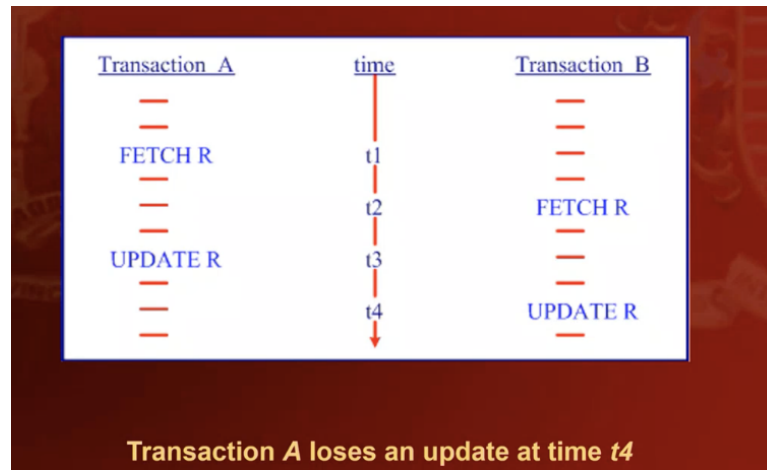
- BIJ must be big enough
- worst case, in dbspace also not there (FIFO scheme)
- T_i must be roll back.

If the old version can't be found, not even in the BIJ, T_i get rollbacked.

- (error from oracle) Error: Serializability cannot be achieved. → solution enlarge old value area
- Oracle, postgres used half locking half timestamp.
 - to solve prob 3, use multiversion → A never wait (know which version should read)
 - multiplegranularity → have to wait, but for multiversion, read only transaction never wait
 - insertion , what if B insert Acc 4 → original timestamp roll back → multiversion B can insert anything, A won't read (look at time, know which version need to read)

- 3 and 4 solved using multiversion

Prob1 → loss update



- even multiversion timestamp order → A get rolledback

Prob2 → need commit bit or x lock

Prob3 - 4 → multiversion, A don't have to wait

What else can we do to avoid rolled back

- Use X lock to solved second problem (big company)
- prob1 use lock
- prob 1-2 simply use strict or rigorous 2PL (no deadlock, a little wait)
 - when we use locking → X lock from beginning avoid deadlock

2 technique describe in a book

- 1) Multiversion timestamp ordering
 protocol : prob 1; TA is rolled back
 prob 2; commit bit, prob 3 & 4: multiversion
- 2) Multiversion 2-phase locking
 Prob 1 & 2: strict or rigorous 2 PL
 Prob 3 & 4: multiversion

- first one don't employ any lock



Multiversion Timestamp Ordering

- Each data item Q has a sequence of versions $\langle Q_1, Q_2, \dots, Q_m \rangle$. Each version Q_k contains three data fields:
 - **Content** -- the value of version Q_k .
 - **W-timestamp(Q_k)** -- timestamp of the transaction that created (wrote) version Q_k .
 - **R-timestamp(Q_k)** -- largest timestamp of a transaction that successfully read version Q_k .



Multiversion Two-Phase Locking

- Differentiates between read-only transactions and update transactions
- **Update transactions** acquire read and write locks, and hold all locks up to the end of the transaction. That is, update transactions follow rigorous two-phase locking.
 - Read of a data item returns the latest version of the item
 - The first **write** of Q by T_i results in the creation of a new version Q_i of the data item Q written
 - W-timestamp(Q_i) set to ∞ initially
 - When update transaction T_i completes, commit processing occurs:
 - Value **ts-counter** stored in the database is used to assign timestamps
 - **ts-counter** is locked in two-phase manner
 - Set $TS(T_i) = \text{ts-counter} + 1$
 - Set W-timestamp(Q_i) = $TS(T_i)$ for all versions Q_i that it creates
 - **ts-counter** = **ts-counter** + 1